Venue-Connect AI Chatbot Guide

Complete Technical Blueprint, Architecture, & Step-by-Step Execution Guide

1. Project Background & Stack

Venue-Connect is a digital marketplace connecting venue seekers with venue owners across Zimbabwe. The AI Chatbot (VenueBot) allows seekers to search, filter, and compare venues using natural language queries, leveraging Python, Django REST Framework, MySQL, React, and Google Gemini LLM API.

Component	Technology	Purpose
Frontend	React (JSX + CSS)	Interactive Chat UI & Venue Cards
Backend API	Python / Django / DRF	REST Endpoints & Business Logic
Database	MySQL / PostgreSQL	Relational Venue & Booking Schemas
AI Engine	Google Gemini 2.5/3.6 Flash	JSON Entity Extraction & NLG Replies

2. Request Lifecycle (17-Stage Breakdown)

Example Query: *'Find me a wedding venue in Harare for 200 people under $600.'*

1. User enters natural language message into React input component.
2. React captures input state and appends user bubble to message history.
3. React dispatches HTTP POST payload { message, session_id } to Django API.
4. Django WSGI/ASGI router routes request to chatbot API View.
5. Django REST Framework validates input payload using ChatRequestSerializer.
6. ChatbotService orchestrates conversation context, LLM extraction, and DB search.
7. Google Gemini LLM analyzes message context and identifies intent as 'search_venue'.
8. Gemini extracts structured parameters (Harare, Wedding, 200 guests, $600 budget).
9. Django parses structured JSON returned from Gemini into a Python dictionary.
10. VenueSearchService constructs dynamic Django ORM Q() query filters.
11. Django ORM executes SQL SELECT query against active MySQL database records.
12. RecommendationService ranks matching venues based on budget and capacity fit.
13. Top matching venue metadata are formatted into LLM context text.
14. Gemini generates natural language response text summarizing top recommendations.
15. Django serializes response into JSON payload with session ID & venue array.

16. React receives HTTP 200 response and appends assistant response bubble.

17. React renders interactive Venue Cards with price, rating, and 'Book Venue' button.

3. Django Database Models (models.py)

The relational database schema supports seekers, venue owners, availability, reviews, and persistent chat sessions:

• **User (AbstractUser)**: Custom auth model with seeker, owner, and admin role choices.

• **VenueOwner**: One-to-One extension of User storing company details and contact numbers.

• **EventType & Facility**: Lookup tables for event categories and amenities (WiFi, Parking, Generator).

• **Venue**: Core model storing capacity, price_per_day, location, and composite DB indexes.

• **VenueAvailability**: Date-level availability schedule (is_available=True/False).

• **Booking & Review**: Reservation tracking and 1-5 star user reviews.

• **ChatSession & ChatMessage**: UUID session tracking & JSON structured conversation history.

4. How to Run the Chatbot (Step-by-Step)

Backend Setup (Django REST Framework)

```
# 1. Navigate to backend directory
cd venue_connect\backend

# 2. Create and activate virtual environment
python -m venv venv
.\venv\Scripts\Activate.ps1

# 3. Install requirements
pip install -r requirements.txt

# 4. Set Gemini API Key
$env:GEMINI_API_KEY="your_gemini_api_key_here"

# 5. Run migrations & seed test data
python manage.py makemigrations
python manage.py migrate
python seed_data.py

# 6. Start Django Server
python manage.py runserver 8000
```

Frontend Setup (React)

```
# 1. Navigate to frontend directory
cd venue_connect\frontend

# 2. Start React development server
npm run dev
# Open browser at http://localhost:5173
```